Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Арифметические операции с полиномами»

Выполнил:
студент группы 3824Б1ФИ2
Старостин Д.Д.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 8
3.1. Описание структуры программы 8
3.2. Описание структур данных 9
3.3. Описание алгоритмов 12
4. Результаты экспериментов и тестирования 14
Заключение 16
Литература 17

Введение

Полиномы являются одним из базовых объектов алгебры и широко используются в различных областях математики, информатики и инженерных расчётов. Они применяются при моделировании физических процессов, решении алгебраических уравнений, аппроксимации функций и разработке численных методов. Во многих вычислительных задачах возникает необходимость выполнять арифметические операции над полиномами, такие как сложение, вычитание и умножение.

При разработке программных средств для работы с полиномами важную роль играет выбор структуры данных для их хранения и эффективная реализация алгоритмов обработки. Представление полинома в виде набора мономов позволяет выполнять операции над ним, объединяя и преобразуя отдельные элементы выражения. При этом необходимо учитывать особенности математической структуры полинома, например упорядоченность мономов и приведение подобных слагаемых.

Реализация подобных алгоритмов представляет интерес с точки зрения изучения структур данных и методов обработки математических выражений.

1. Постановка задачи

Цель работы — Разработать программу, выполняющую арифметические операции с полиномами трех переменных (x, y и z): сложение, вычитание, умножение на константу, умножение двух полиномов. Считается, что полином составлен из мономов от трех переменных с ограничением на степень каждой переменой от 0 до 9. Коэффициенты полинома - вещественные числа. В ходе выполнения задачи предполагается проверка работоспособности программы с помощью Google Test-ов. Кроме того, необходимо разработать пользовательское консольное приложение.

2. Руководство пользователя

При запуске приложения пользователя встречает меню. С помощью меню можно производить операции с полиномами из списка добавленных пользователем, взаимодействовать со списком полиномов и выйти из программы. Взаимодействие с меню происходит посредством ввода числа, ассоциированного с одним из пунктов меню. При неверном вводе выводится ошибка и предоставляется возможность повторного ввода.

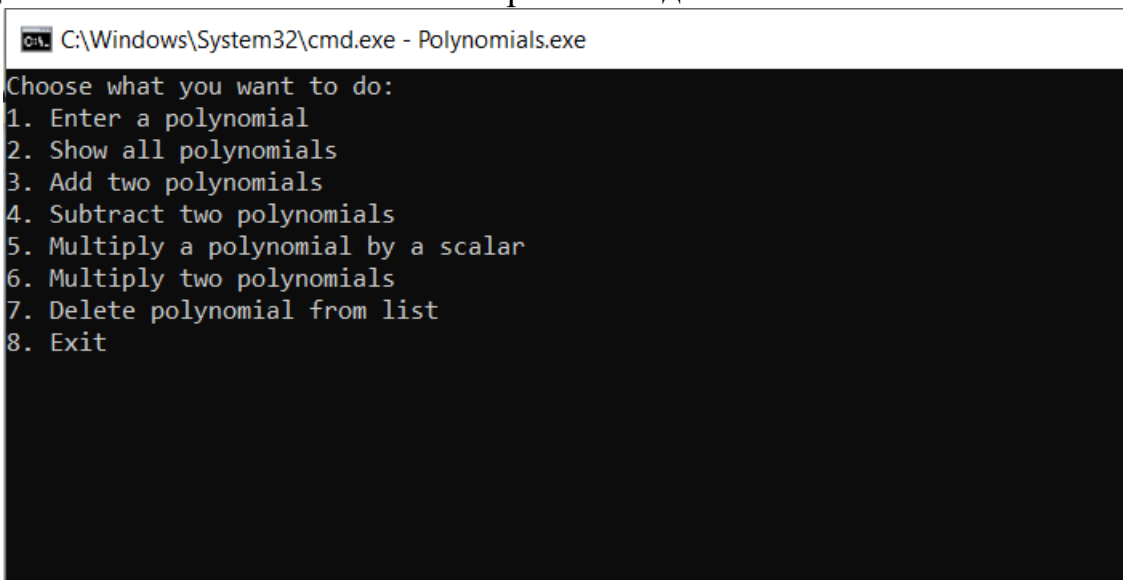

Рис.1 Меню

Описание пунктов меню:

1. «Enter a polynomial». Добавить новый полином в список. Пользователю предлагается ввести полином, приведено упрощённое описание формата и пример. Формат:
   - Мономы разделяются пробелами
   - Между мономами должен стоять + или –
   - Формат монома: минимум 3 символа. 3 последние символа отвечают за степень, а предшествующие им – за вещественный коэффициент. Коэффициент не отделяется от степеней. Допустимые символы: + или – для знака, одна точка для вещественных чисел, цифры.
   - Коэффициент может иметь собственный знак

   При неверном вводе выводится сообщение об ошибке с кратким описание ошибки.

```
C:\Windows\System32\cmd.exe - Polynomials.exe
Enter a polynomial. Format: Space-separated monomials: CXYZ,
where C is the coefficient and X, Y, Z are degrees.

Optional '+' or '-' may precede each monomial. Example: 1111 - 231 + -12453 - 123
```

Рис.2 Ввод полинома

2. «Show all polynomials». Вывод списка доступных полиномов, введённых пользователем.

3. «Add two polynomials». Сложение двух полиномов из списка. При выборе пункта программа предлагает вывести список доступных полиномов. Далее пользователь должен выбрать первый и второй операнд. После выполнения операции пользователь имеет возможность добавить результат в список полиномов. Если пользователь решил сохранить, то ему будет показано, под каким номером сохранён результат.

```
C:\Windows\System32\cmd.exe - Polynomials.exe
1. 1.000000*x^1y^2z^3 + 1.000000*x^5y^4z^7
2. -5.000000*x^1y^2z^3 + 1.000000*x^3y^2z^1 + 0.100000*x^4y^5z^6
Enter the number of the first polynomial: 1
1.000000*x^1y^2z^3 + 1.000000*x^5y^4z^7
Enter the number of the second polynomial: 2
-5.000000*x^1y^2z^3 + 1.000000*x^3y^2z^1 + 0.100000*x^4y^5z^6
The result of the addition is: -4.000000*x^1y^2z^3 + 1.000000*x^3y^2z^1 + 0.100000*x^4y^5z^6 + 1.000000*x^5y^4z^7
You want to save it? 1.Yes 2. No
```

Рис.3 Сложение полиномов

4. «Subtract two polynomials». Вычитание двух полиномов. Функционал аналогичен описанному в пункте 3.
5. «Multiply a polynomial by a scalar». Умножение полинома из списка на число. При выборе пункта программа предлагает вывести список доступных полиномов. Далее пользователь должен выбрать полином из списка и ввести вещественное число. После выполнения операции пользователь имеет возможность добавить результат в список полиномов. Если пользователь решил сохранить, то ему будет показано, под каким номером сохранён результат.
6. «Multiply two polynomials». Умножение двух полиномов из списка. При выборе пункта программа предлагает вывести список доступных полиномов. Далее пользователь должен выбрать первый и второй операнд. После выполнения операции пользователь имеет возможность добавить результат в список

полиномов. Если пользователь решил сохранить, то ему будет показано, под каким номером сохранён результат. Если в результате умножение получится моном со степенью больше 9, будет выведено сообщение об ошибке.

```
C:\Windows\System32\cmd.exe - Polynomials.exe
1. 1.000000*x^1y^2z^3 + 1.000000*x^5y^4z^7
2. -5.000000*x^1y^2z^3 + 1.000000*x^3y^2z^1 + 0.100000*x^4y^5z^6
3. -4.000000*x^1y^2z^3 + 1.000000*x^3y^2z^1 + 0.100000*x^4y^5z^6 + 1.000000*x^5y^4z^7
Enter the number of the first polynomial: 2
-5.000000*x^1y^2z^3 + 1.000000*x^3y^2z^1 + 0.100000*x^4y^5z^6
Enter the number of the second polynomial: 3
-4.000000*x^1y^2z^3 + 1.000000*x^3y^2z^1 + 0.100000*x^4y^5z^6 + 1.000000*x^5y^4z^7
Error! when multiplying, the power of one of the variables turned out to be greater than 9
Для продолжения нажмите любую клавишу . . .
```

Рис.4 Ошибка при умножении

7. «Delete polynomial from list». Удаление полинома из списка. При выборе пункта программа предлагает вывести список доступных полиномов. Далее пользователь должен ввести номер удаляемого полинома. Если номера нет в списке, выводится ошибка.
8. «Exit». Выход из программы.

3. Руководство программиста

3.1. Описание структуры программы

Программа реализует работу с полиномами от трёх переменных. Основной функционал включает создание полиномов, выполнение арифметических операций над ними, вывод и получения строкового представления полинома.

Далее описаны модули, представленные в программе.

Модуль Polynomial.h

Данный модуль содержит объявления всех классов, структур и методов, используемых для представления и обработки полиномов. В модуле объявлен основной класс **Polynomial** – класс, представляющий полиномы. В классе перегружены арифметические операторы, реализованы методы очистки полинома, получения строкового представления полинома, получения константных итераторов, а также метод swap. В приватной области реализованы методы для поиска и вставки. Реализована перегрузка оператор в стандартный поток ввода и вывода.

Класс Polynomial содержит вложенные структуры и классы:

- **Monomial**. Структура, представляющая мономы. Хранит информацию о степени и коэффициенте монома. В структуре перегружены все необходимые операторы.
- **Node**. Структура, представляющая узел односвязного списка.
- **iterator**. Класс итератора, используемый для упрощения внутренней реализации, приватный.
- **const_iterator**. Класс константного итератора. Предназначен для внешнего обхода элементов без возможности их изменения.

Модуль Polynomial.cpp

Модуль **Polynomial.cpp** содержит реализации всех методов, объявленных в заголовочном файле.

В нём реализованы:

- конструкторы и деструктор класса
- методы добавления и удаления элементов списка
- арифметические операции над мономами и полиномами
- поиск в списке
- реализация итераторов
- получение строкового представления полинома
- ввод и вывод

Модуль console_app.cpp

Модуль **console_app.cpp** содержит реализацию консольного приложения. В нём реализованы меню, ввод в программу и вывод на экран полиномов, арифметические операции.

Модуль Polynomials_tests

Для проверки корректности работы программы был разработан отдельный модуль тестирования. Тестирование реализовано с использованием библиотеки Google Test.

Модуль тестирования содержит набор тестов, проверяющих корректность работы класса Polynomial, включая конструкторы, арифметические операции, методы управления состоянием объекта, операции ввода-вывода.

3.2. Описание структур данных

Класс Polynomial

Класс Polynomial предназначен для представления полинома, состоящего из набора мономов. Полином хранится в виде односвязного списка с фиктивной «головой», элементы которого упорядочены по возрастанию степеней.

Поля класса:

- **Node* head**. Указатель на фиктивный головной элемент списка. Используется для упрощения операций вставки и удаления.
- **Node* tail**. Указатель на последний элемент списка. Используется для быстрой вставки в конец.

Методы класса:

- **Polynomial()**. Конструктор по умолчанию. Выделяет память для head и устанавливает tail на head.
- **Polynomial(const double& src)**. Конструктор преобразования из вещественного числа в полином. Создаёт полином нулевой степени с одним мономом с переданным коэффициентом.
- **Polynomial(const std::string& src)**. Конструктор создаёт полином на основе строкового представления. Из переданной строки извлекаются мономы. При успешном создании монома он добавляется в полином. Иначе, при передаче некорректной строки генерируется исключение.
- **Polynomial(const Polynomial& other)**. Конструктор копирования. Создаётся полином, копия переданного. Для этого выполняется последовательный обход списка мономов исходного полинома и создание новых узлов с теми же значениями.
- **~Polynomial()**. Деструктор. Освобождает выделенную память. Использует метод clear().
- **void push_back(const Monomial& src)**. Метод добавляет новый моном в конец списка. Создаётся новый узел, который становится последним элементом списка. Указатель tail обновляется.
- **void insert_after(const iterator& pos, const Monomial& src)**. Метод выполняет вставку нового узла после указанной позиции. Если следующий узел содержит моном с такой же степенью, коэффициенты суммируются.
- **iterator find(const Monomial& monomial)**. Метод выполняет поиск позиции, после которой может быть вставлен новый моном не нарушая порядок. Поиск выполняется последовательным обходом списка.

- **void insert(const Monomial& src)**. Метод добавляет новый моном в полином с сохранением порядка элементов. Использует методы find и insert_after.
- **void clear()**. Удаляет все элементы списка, кроме head.
- **void swap(Polynomial& other)**. Метод выполняет обмен содержимым двух объектов класса Polynomial.
- **std::string get(bool format=0) const**. Метод возвращает строковое представление полинома в одном из форматов. Формат по умолчанию – компактный формат, отражающий степени и коэффициенты. Другой формат – более читаемый для человека, представляет математическую запись.
- **iterator begin()**. Приватный метод. Возвращает итератор, указывающий на первый элемент списка.
- **const_iterator begin() const**. Публичный метод. Возвращает константный итератор, указывающий на первый элемент списка.
- **iterator end()**. Приватный метод. Возвращает итератор, указывающий на элемент списка, следующий за последним.
- **const_iterator end() const**. Публичный метод. Возвращает константный итератор, указывающий на элемент списка, следующий за последним.
- **iterator before_begin()**. Приватный метод. Возвращает итератор, указывающий на фиктивный узел списка head.
- **const_iterator before_begin() const**. Публичный метод. Возвращает константный итератор, указывающий на фиктивный узел списка head.
- **Polynomial& operator=(const Polynomial& other)**. Копирование содержимого одного полинома в другой.
- **Polynomial operator-() const**. Унарный минус. Возвращает полином с обратными по знаку коэффициентами.
- **Polynomial operator+(const Polynomial& other) const**, **Polynomial& operator+=(const Polynomial& other)**. Сложение двух полиномов. Выполняется с помощью алгоритма слияния двух полиномов.
- **Polynomial operator-(const Polynomial& other) const**, **Polynomial& operator-=(const Polynomial& other)**. Вычитание двух полиномов. Выполняется с помощью алгоритма слияния двух полиномов.
- **Polynomial operator*(const double& scalar) const**. Умножение полинома на скаляр.
- **Polynomial operator*(const Polynomial& other) const**, **Polynomial& operator*=(const Polynomial& other)**. Умножение двух полиномов.

Класс Monomial

Структура, представляющая мономы. Хранит информацию о степени и коэффициенте монома.

Поля класса:

- **double coef**. Коэффициент монома.
- **int degs.** Набор степеней монома.

Метода класса:

- Конструкторы: по умолчанию, преобразования из вещественного числа, из строки, копирования.
- **Monomial operator*(const Monomial& other) const**, **Monomial& operator*=(const Monomial& other)**. Умножение мономов. Если при умножении степень результирующего монома превышает 9 генерируется исключение.
- Также реализованы арифметические операторы: сложения (+, +=), вычитания (-, -=), унарный минут
- Операторы сравнения: <, >, ==, != . Сравнение происходит по степеням и используется для поддержания порядка.

Структура Node

Структура представляет узел односвязного списка.

Поля:

- **Monomial data**. Моном.
- **Node* next**. Указатель на следующий узел.

Конструктор: Node(const Monomial& src=Monomial(), Node* next = nullptr). Создаёт узел списка, в поле data копируется значение аргумента src, в поле next – next.

Итераторы

Для обхода элементов полинома реализованы классы iterator и const_iterator. Так как в полиноме соблюдается порядок, вне класса доступен только const_iterator.

Каждый итератор содержит указатель на текущий узел списка.

Конструктор принимает указатель на узел.

Реализованные методы:

- **Monomial& operator*()**. Разыменование. Возвращает ссылку на текущий моном.
- **Monomial* operator->()**. Предоставляет доступ к полям текущего монома.
- **iterator& operator++()**. Переход к следующему по порядку элементу списка.
- **bool operator==(const iterator& other) const**, **bool operator!=(const iterator& other) const**. Сравнение итераторов.

Ввод и вывод

Перегружены операторы << и >>. Вывод осуществляет в математическом формате. Ввод осуществляется в формате, описанном выше для пользователей.

Глобальные переменные

static const double EPS = 0.000'001 – переменная для оценки близости к нулю вещественных чисел.

3.3. Описание алгоритмов

- **Вставка мономов**. void insert(const Monomial& src).
  1) Выполняется линейный поиск позиции для вставки.
  2) Если найденная позиция – конец списка, выполняется push_back.
  3) Если на найденной позиции моном с такой же степенью, коэффициенты вставляемого и найденного мономов суммируются.
  4) Если моном с такой степенью не найден, то создаётся номой узел и вставляется в позицию без нарушения порядка.

  Сложность: O(n), где n – количество мономов в полиноме. Обоснование: линейный проход по списку для поиска.

- **Вставка после итератора**.
  1) Если переданная позиция – конец списка, выполняется push_back.
  2) Если на переданной позиции моном с такой же степенью, коэффициенты вставляемого и переданного мономов суммируются.
  3) Если моном со степенью как у переданного нет в списке, то создаётся номой узел и вставляется в позицию без нарушения порядка.

  Сложность: O(1). Обоснование: поиск не производится, описанные операции выполняются за O(1).
- **Сложение полиномов**. Реализовано с помощью алгоритма слияния двух упорядоченных списков. Результат сохраняется в первом списке.
  1) Создаются итераторы для обхода it_l (первый список), it_r (второй список), и итератор prev (первый список) для вставки и удаления.
  2) Пока один из итераторов не достиг конца выполняется обход со сравнение текущих мономов.
  3) Возможные случаи:
     a) Степень первого монома меньше. it_l и prev продвигаются вперёд.
     b) Степень второго монома меньше. Текущий моном второго списка вставляется в первый после prev.
     c) Степени мономов равны. Коэффициенты суммируются. Если сумма коэффициентов меньше EPS, то моном из it_l удаляется из списка, а из it_r не вставляется в него.
  4) При достижении конца первого списка в его конец вставляются оставшиеся полиномы из второго.

  Сложность: O(n + m), n – количество мономов в 1-м списке, m – во 2-м. Обоснование: производится полный проход по всем элементам обоих списков.

- **Умножение двух полиномов**. Алгоритм:
  1) Для каждого монома первого полинома выполняется умножение на каждый моном второго полинома. Полученные мономы формируют промежуточный полином.
  2) Промежуточный полином добавляется к итоговому полиному с помощью операции сложения.

Сложность: $O(n^2m)$, n – число мономов в первом полиноме, m – во втором. Обоснование: рассмотрим внешний цикл. Умножение мономов выполняется за O(1) – m умножений. Далее добавляем к res – (i - 1)m + m операций. Суммируем по i от 1 до n: $\sum_{i=1}^{n}(i-1)m = \frac{m(n-1)}{2} * n = O(mn^2)$. Получаем $mn^2 + mn = O(mn^2)$

- **Очистка полинома.** void clear(). Алгоритм:
  1) Список обходится последовательно начиная с head->next.
  2) Каждый узел освобождается.
  3) В итоге остаётся пустой список с фиктивным head.

  Сложность: O(n), n – размер полинома. Обоснование: последовательный обход всего списка.

4. Результаты экспериментов и тестирования

Для проверки корректности работы реализованного класса Polynomial было проведено модульное тестирование программы. Тестирование выполнялось с использованием библиотеки Google Test, которая позволяет автоматически проверять корректность работы отдельных компонентов программы.

Были проведены следующие группы тестов:

1. **Polynomial_constructor**. В ходе тестов была проверена корректность работы всех конструкторов класса Polynomial.

   Были проведены следующие тесты:
   - Конструктор по умолчанию. Проверялось корректное создание пустого полинома без генерации исключений.
   - Конструктор из числового значения. Проверялось, что создаётся полином, состоящий из одного монома, коэффициент которого равен переданному значению, а степени переменных равны нулю. Также было проверено отсутствие исключений.
   - Конструктор из строкового представления. Проверялось корректное создание полинома на основе строки, содержащей последовательность мономов.
   - Обработка некорректных входных данных. Проверялась генерация исключений при передаче строк с неверным форматом.
   - Конструктор копирования. Проверялось корректное создание копии существующего полинома.
2. **Polynomial_operators**. Была проведена серия тестов для проверки корректности реализации арифметических операций над полиномами и других операторов.

   Проверялись следующие операции:
   - Унарное отрицание полинома
   - Сложение полиномов
   - Вычитание полиномов
   - Умножение полинома на скаляр
   - Умножение двух полиномов
   - Копирующее присваивание

   В ходе тестирования проверялись следующие случаи:
   - Операции над полиномами, содержащими несколько мономов
   - Операции над пустыми полиномами
   - Операции, приводящие к обнулению коэффициентов
   - Корректность объединения мономов с одинаковыми степенями
   - Возникновение переполнения при умножении
   - Присваивание и самоприсванивание
3. **Polynomial_methods**. Тестирование методов управления объектом. Отдельная серия тестов была посвящена проверке методов управления состоянием объекта.

   Были проверены следующие методы:
   - clear() — очистка полинома

- swap() — обмен содержимого двух полиномов

Таким образом, в ходе тестов были проверены различные граничные случаи работы программы:

- Операции с пустыми полиномами
- Обнуление полинома в результате арифметических операций и мономов
- Корректная работа объекта после вызова метода clear()
- Операции умножения, приводящие к переполнению степеней

Проведённые тесты показали, что программа корректно обрабатывает все рассмотренные ситуации.

Тестирование показало, что реализованные алгоритмы корректно выполняют все предусмотренные операции над полиномами. В результате проведённых тестов не было выявлено ошибок, приводящих к некорректной работе программы.

Использование автоматизированного тестирования позволило:

- Повысить надёжность программной реализации
- Проверить корректность работы алгоритмов
- Убедиться в правильности обработки граничных ситуаций

Подводя итог, результаты тестирования подтверждают корректность реализации класса Polynomial и соответствие программы поставленным требованиям.

Заключение

В ходе выполнения данной работы была разработана программа для работы с полиномами от трёх переменных. Основной целью работы являлась реализация структуры данных для хранения полинома и алгоритмов выполнения арифметических операций над полиномами.

В процессе разработки был реализован класс Polynomial, представляющий полином в виде упорядоченного односвязного списка мономов. Для описания отдельного монома была разработана структура Monomial, содержащая коэффициент и степени переменных.

В рамках работы были реализованы следующие возможности программы:

- создание полиномов различными способами (конструктор по умолчанию, конструктор из числа, конструктор из строки)
- реализация арифметических операций над полиномами (сложение, вычитание, умножение полиномов, умножение на скаляр)
- реализация операторов присваивания и унарных операций;
- реализация итераторов для обхода элементов полинома
- реализация методов управления структурой данных (вставка элементов, очистка полинома, обмен содержимым объектов)
- генерация исключений при возникновении ошибок, связанных с некорректными входными данными или переполнением степеней

Для проверки корректности работы программы был разработан модуль автоматического тестирования с использованием библиотеки Google Test. В ходе тестирования были проверены конструкторы класса, операторы, обработка исключительных ситуаций и методы управления объектами. Результаты тестирования показали корректность работы реализованных алгоритмов и устойчивость программы при различных входных данных.

Также было разработано консольное приложение, обеспечивающее взаимодействие программы с пользователем. Приложение реализует текстовое меню, позволяющее вводить и удалять полиномы, выполнять арифметические операции над ними и выводить результаты вычислений.

Таким образом, поставленные задачи были успешно выполнены, реализованная программа корректно выполняет операции над полиномами.

Литература

1. Сайт cppreference: https://en.cppreference.com/